# Sistema de Moeda Estudantil

## Arquitetura, camada de persistência e tutorial das tecnologias

Laboratório 3 | Projeto de Software | PUC Minas

Sprint 03 | 2º semestre de 2026


**Integrantes**

Fabrício Rocha Lopes

Gabriel Silva Neiva Amaro

Otávio Chaves Silva

Samuel Rocha Ferraz Gonçalves Rebula

**Professora**

Profa. Milena Menezes Adão

# O problema: reconhecer o mérito com uma moeda virtual

### Aluno
Cadastra-se, recebe moedas, consulta o extrato e resgata vantagens.

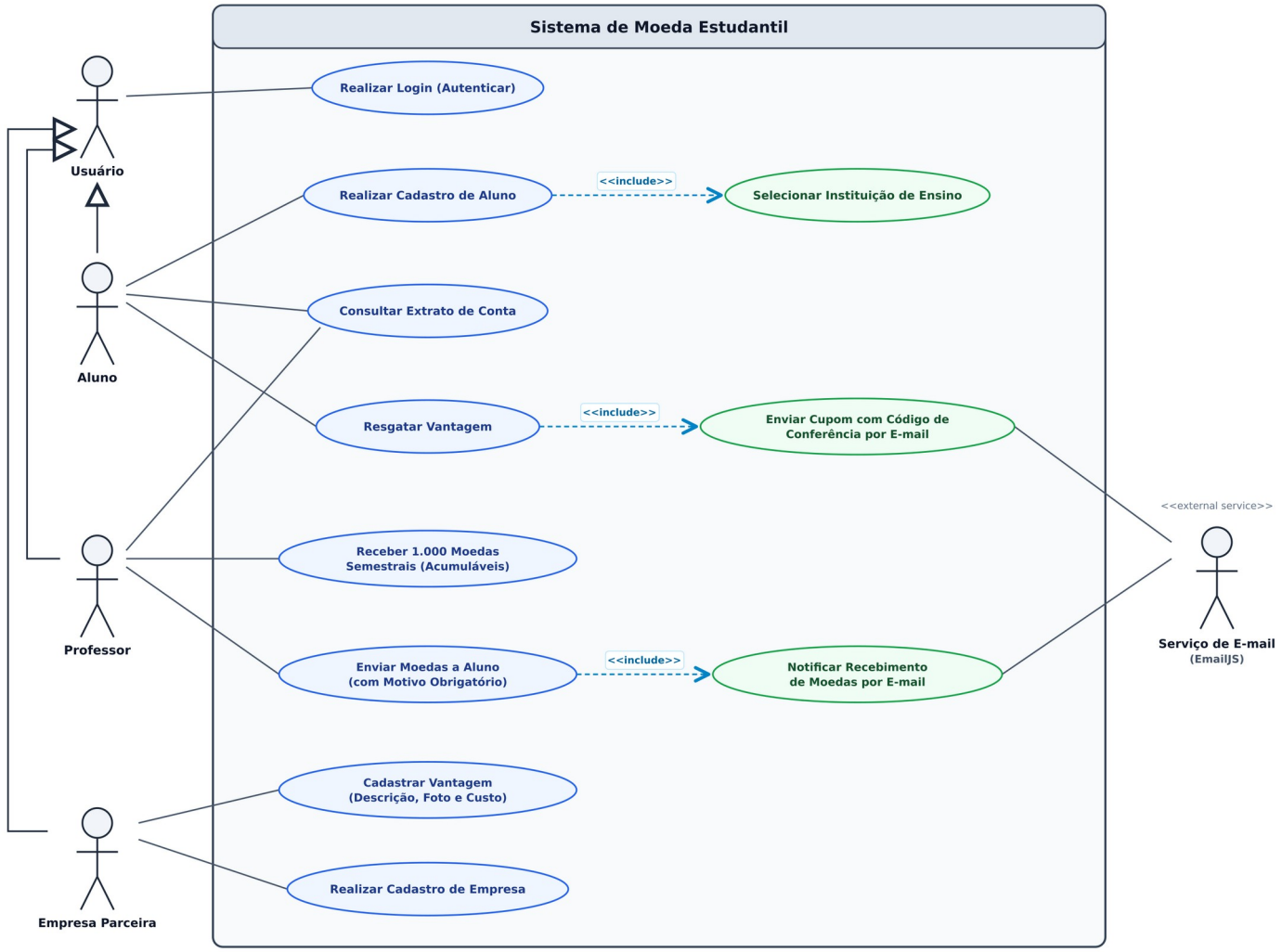
### Professor
Pré-cadastrado; recebe 1.000 moedas por semestre e as envia aos alunos.

### Empresa parceira
Cadastra-se e oferece vantagens com descrição, foto e custo em moedas.

## Regras-chave

- **1.000 moedas por semestre,** acumuláveis: o que sobra soma com o novo crédito.
- **Envio de moedas:** saldo suficiente e motivo obrigatório; o aluno é avisado por e-mail.
- **Resgate de vantagem:** desconta o saldo e gera um código único de conferência.
- **E-mails do cupom:** um para o aluno e outro para a empresa, ambos com o código.
- **Login e senha** para aluno, professor e empresa, com autenticação em todas as operações.

# Casos de uso


Sistema de Moeda Estudantil
Usuário
Aluno
Professor
Empresa Parceira
Realizar Login (Autenticar)
Realizar Cadastro de Aluno
<<include>>
Selecionar Instituição de Ensino
Consultar Extrato de Conta
Resgatar Vantagem
<<include>>
Enviar Cupom com Código de Conferência por E-mail
Receber 1.000 Moedas Semestrais (Acumuláveis)
Enviar Moedas a Aluno (com Motivo Obrigatório)
<<include>>
Notificar Recebimento de Moedas por E-mail
Cadastrar Vantagem (Descrição, Foto e Custo)
Realizar Cadastro de Empresa
<<external service>>
Serviço de E-mail
(EmailJS)

# Arquitetura MVC: visão geral

### View
React SPA com Vite: AuthView, AlunoView, ProfessorView e EmpresaParceiraView.

### Controller
Spring Web REST: rotas HTTP, validação do payload e autenticação (token Bearer).

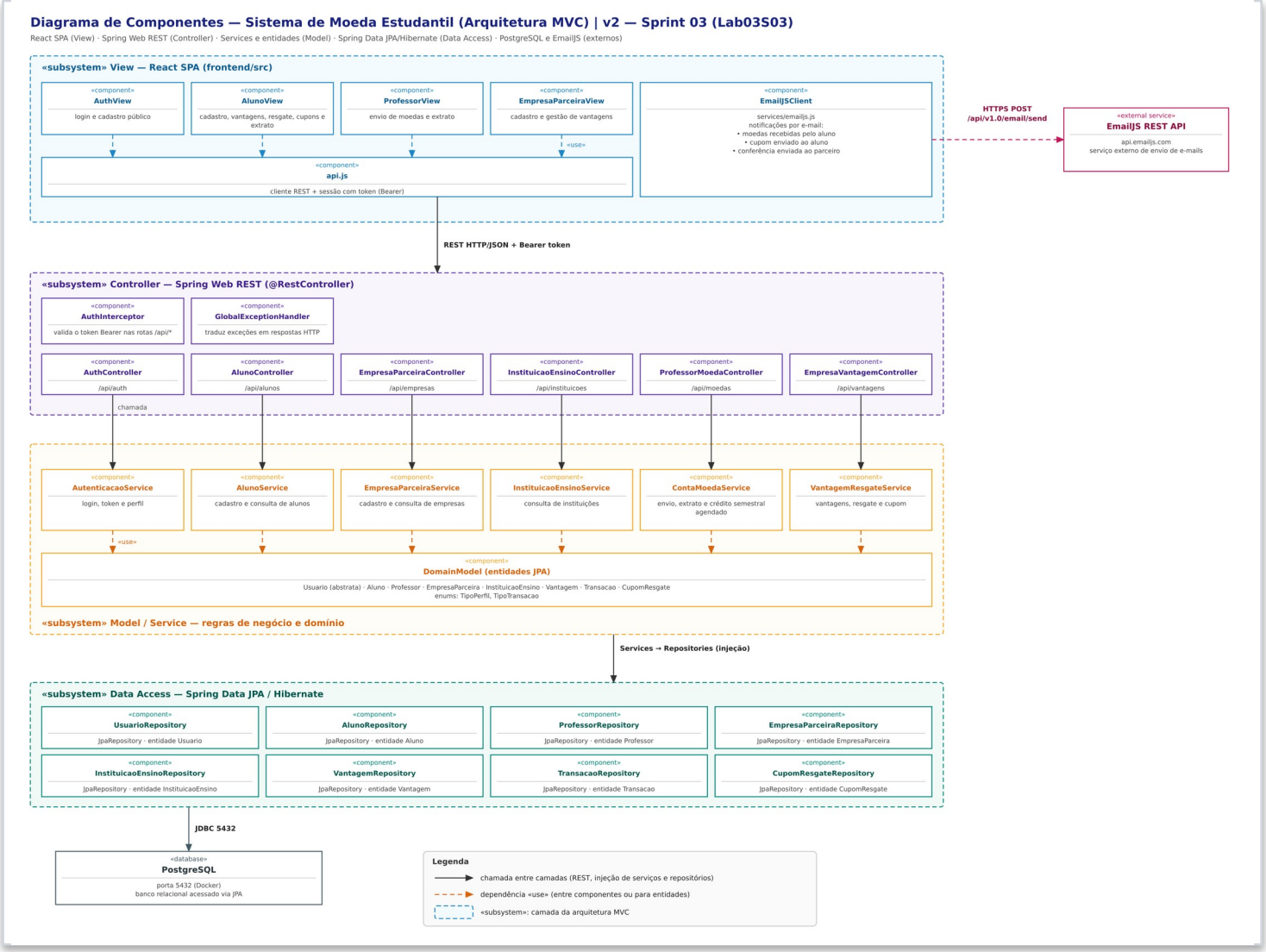
### Model
Entidades JPA (regras de negócio) e services: ContaMoedaService, VantagemResgateService.

### Persistência
Spring Data JPA + Hibernate sobre PostgreSQL 16.

### E-mail
EmailJS, chamado pelo front-end via REST/HTTPS (aviso de moedas e cupom).

### Execução
Docker Compose: postgres, backend e frontend com um único comando.

# Diagrama de componentes




- **View:** telas React por perfil.
- **Controller:** rotas REST e AuthInterceptor.
- **Model:** services e entidades JPA.
- **Persistência:** repositórios e PostgreSQL.
- **E-mail:** EmailJS via REST.

# Camada Controller: rotas REST e autenticação

| Grupo | Rotas (base /api) | Acesso |
|---|---|---|
| **auth** | `POST /auth/login, /auth/logout` | login público |
| **alunos** | `GET, POST, PUT, DELETE /alunos[/{id}]` | POST público |
| **empresas** | `GET, POST, PUT, DELETE /empresas[/{id}]` | POST público |
| **instituicoes** | `GET /instituicoes` | público |
| **moedas** | `GET /moedas/extrato, POST /moedas/envios` | aluno/prof. |
| **vantagens** | `CRUD /vantagens, /{id}/resgates, /cupons` | logado |

## AuthInterceptor

Exige o cabeçalho
**`Authorization: Bearer <token>`**

**Rotas públicas:**

- login
- cadastro de aluno e empresa
- lista de instituições

Sem token válido: 401 com {"erro": ...}

**Erros em JSON padronizado:** 400 regra de negócio, 401 sem sessão, 403 perfil errado, 404 não encontrado, 409 vínculo existente.

# Camada Model: as regras de negócio

### Professor.enviarMoedas

Exige saldo suficiente e motivo; desconta do professor, credita o aluno e grava a Transacao.

### Crédito semestral

@Scheduled em 1º/jan e 1º/jul: soma 1.000 moedas ao saldo de cada professor, sem zerar.

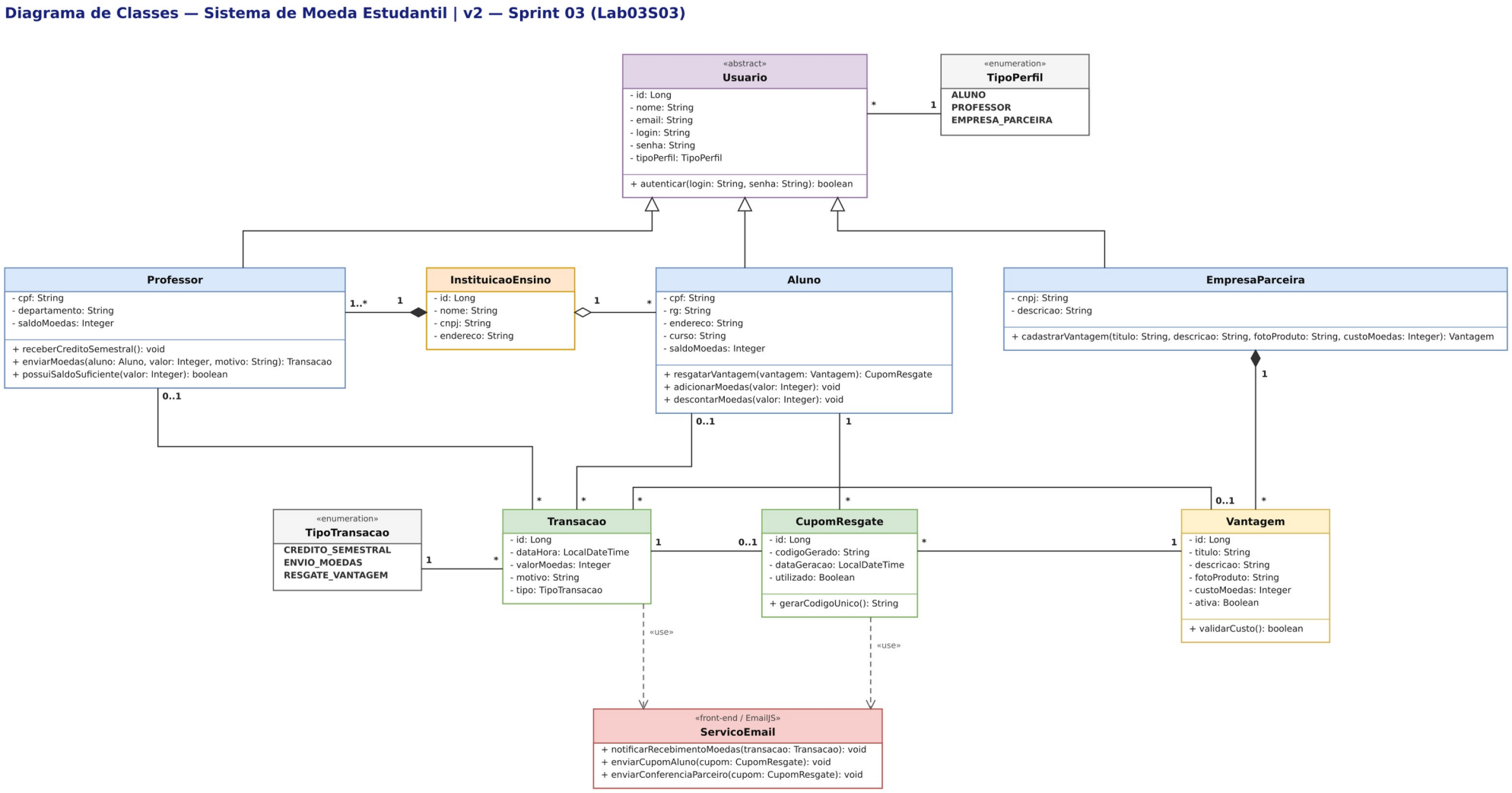
### Aluno.resgatarVantagem

Desconta o custo do saldo e cria o CupomResgate com o código de CupomResgate.gerarCodigoUnico.

### Exclusão lógica de vantagem

DELETE marca ativa = false; a vantagem some do catálogo e os cupons emitidos continuam.

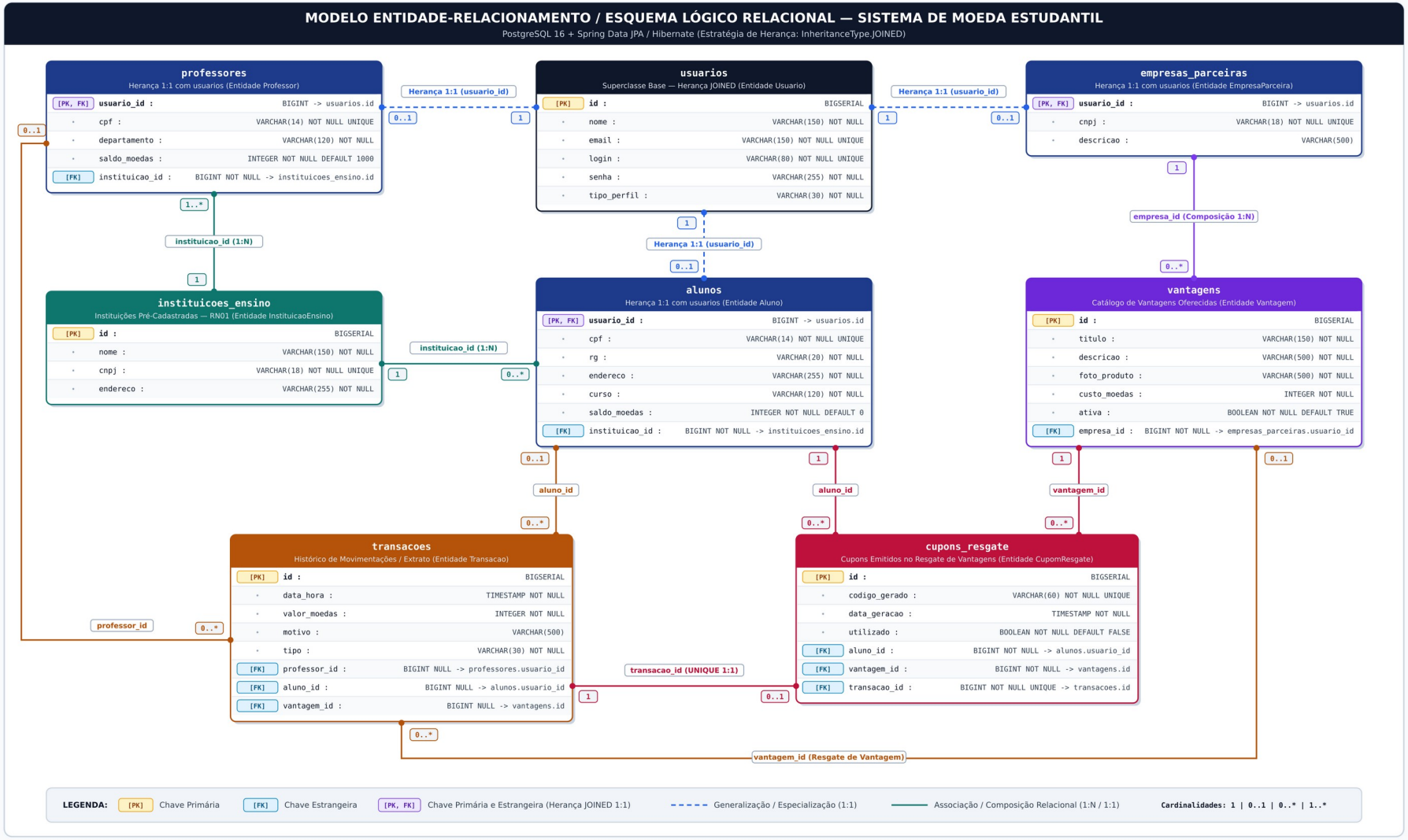
# Diagrama de classes



# Camada de persistência: ORM com Spring Data JPA

- **ORM:** Spring Data JPA + Hibernate sobre PostgreSQL 16, sem SQL escrito à mão.
- **Herança JOINED:** usuarios (base) + alunos, professores e empresas_parceiras, ligadas por usuario_id.
- **8 repositórios** JpaRepository com consultas derivadas (findByCpf, findByLogin, findByAtivaTrue...).
- **ddl-auto=update:** o Hibernate cria e ajusta as tabelas na subida da aplicação.
- **DataSeeder:** carga idempotente de instituições e professores pré-cadastrados (exigência do enunciado).

# Modelo entidade-relacionamento



# Tutorial 1: Spring Boot + JPA

**Entidades (model/Usuario.java, Aluno.java)**

```
@Entity @Table(name = "usuarios")
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Usuario { ... }

@Entity @Table(name = "alunos")
@PrimaryKeyJoinColumn(name = "usuario_id")
public class Aluno extends Usuario {
  @ManyToOne(fetch = FetchType.EAGER)
  @JoinColumn(name = "instituicao_id")
  private InstituicaoEnsino instituicaoEnsino;
}
```

**Repositório (consulta derivada)**

```
public interface TransacaoRepository
    extends JpaRepository<Transacao, Long> {

  List<Transacao>
    findByAlunoIdOrderByDataHoraDesc(
        Long alunoId);
}
```

**SQL gerado pelo Hibernate**

```
select t.* from transacoes t
where t.aluno_id = ?
order by t.data_hora desc
```

**Sem implementar nada:** o nome do método vira a consulta e as anotações viram o esquema do banco.

# Tutorial 2: React + Vite

**Componente (useState + useEffect)**

```
function Extrato() {
  const [dados, setDados] = useState(null);

  useEffect(() => {
    request('/moedas/extrato')
      .then(setDados);
  }, []);

  return <p>Saldo: {dados?.saldoMoedas}</p>;
}
```

**frontend/src/services/api.js (simplificado)**

```
const API_BASE_URL =
  import.meta.env.VITE_API_BASE_URL;

async function request(path, options = {}) {
  const response = await fetch(
    `${API_BASE_URL}${path}`, {
      headers: {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${token}`,
      }, ...options });
  if (!response.ok) throw new Error(...);
}
```

- **Vite:** servidor de desenvolvimento rápido (porta 5173) e build de produção.
- **Uma função por rota:** listarAlunos(), criarAluno(dados)... todas passam pelo request().

# Tutorial 3: Docker Compose + EmailJS

**docker-compose.yml (resumo)**

```
services:
  postgres:   # postgres:16-alpine
    ports: ["5432:5432"]
    healthcheck: pg_isready
  backend:    # maven + Java 21
    command: mvn spring-boot:run
    depends_on: postgres (healthy)
  frontend:   # node:20-alpine
    command: npm run dev (Vite)
    depends_on: backend
```

**EmailJS: chamada REST (variáveis VITE_EMAILJS_*)**

```
fetch('https://api.emailjs.com/api/v1.0/email/send', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    service_id:  VITE_EMAILJS_SERVICE_ID,
    template_id: VITE_EMAILJS_TEMPLATE_ID,
    user_id:     VITE_EMAILJS_PUBLIC_KEY,
    template_params: { to_email, subject, message },
  }),
});
```

**$ docker compose up -d** front em :5173, API em :8080, banco em :5432

# Roteiro da demonstração

### 1 Login do professor

milena.adao / prof123 (saldo 1.000).

### 2 Envio de moedas

Escolhe o aluno, informa o valor e o motivo obrigatório.

### 3 Extrato do aluno

Aluno entra e vê o saldo e a transação recebida.

### 4 Vantagem da empresa

Empresa cadastra título, descrição, custo e foto.

### 5 Resgate

Aluno escolhe a vantagem; o custo é descontado do saldo.

### 6 Cupom e e-mails

Código do cupom na tela e e-mails ao aluno e à empresa.

# Encerramento: aprendizados e próximos passos

## Aprendizados

- Camadas bem separadas facilitam dividir o trabalho.
- Regra de negócio nas entidades, não nos controllers.
- ORM reduz SQL repetitivo, mas é preciso entender o que o Hibernate gera.
- Docker Compose deixa o ambiente reproduzível.

## Evolução dos modelos: v1 para v2

- v1 (Sprint 1): modelagem inicial de classes e componentes.
- v2 (Sprint 3): atributos que faltavam, como Vantagem.ativa e CupomResgate.utilizado.
- Componentes renomeados para os nomes reais do código.
- As duas versões estão no repositório.

**Obrigado! Perguntas?**